LangChain Foundations

A Comprehensive Bootcamp Guide

Chains · Prompt Templates · Memory Systems · Conversation Memory
Buffer Memory · Long-Term Memory · Components Deep Dive

Training Level: Fresher / Entry-Level | Duration: 1-Day Intensive

2025 Edition

Table of Contents

- Token Limits
- Pruning Strategies

Module 8 Intro to Long-Term Memory

- Vector Store Memory
- Entity Memory
- Knowledge Graph Memory
- Combining Memory Types

Module 1 — LangChain Basics

1.1 What is LangChain?

LangChain is an open-source framework designed to simplify the development of applications powered by large language models (LLMs). Created by Harrison Chase in 2022 and rapidly adopted across the AI industry, LangChain provides a unified abstraction layer that lets developers chain together LLM calls, tools, memory systems, and data sources into sophisticated pipelines called — unsurprisingly — chains.

At its core, LangChain solves the problem of orchestration. Raw LLM APIs are stateless and single-turn. Real applications need context, memory, tool access, and multi-step reasoning. LangChain supplies all of this in a composable, production-ready way.

> **■ Core Mission**
> LangChain makes it easy to build context-aware, reasoning applications by connecting LLMs with external data and computation — turning raw model calls into intelligent agents.

1.2 Why LangChain?

- **Abstraction:** Write model-agnostic code — swap OpenAI for Anthropic, Gemini, or Ollama in one line.
- **Composability:** Combine prompts, models, parsers, and memory into reusable pipeline components.
- **Memory:** Give your LLM short-term conversation memory or long-term vector-based recall.
- **Tooling:** Connect LLMs to search engines, databases, APIs, code interpreters, and more.
- **Ecosystem:** 700+ integrations covering vector stores, document loaders, retrievers, and agents.
- **Production-ready:** LangSmith and LangServe handle observability, debugging, and serving.

1.3 Architecture Overview

LangChain follows a layered architecture. At the lowest level sit the model integrations (wrappers around LLM APIs). Above them are the core abstractions — prompts, memory, parsers — which are assembled into chains. Chains can be orchestrated by agents that dynamically decide which tools to call. Finally, LangServe handles API deployment and LangSmith handles tracing and evaluation.

Layer	Components	Purpose
Model I/O	LLMs, Chat Models, Embeddings	Interact with language models
Data Connection	Document Loaders, Splitters, Retrievers	Load & search external data
Chains	LLMChain, Sequential, Router, LCEL	Compose multi-step pipelines
Memory	Buffer, Summary, Vector, Entity	Persist conversation state
Agents	ReAct, Tool-Calling, OpenAI Functions	Dynamic tool-use reasoning
Callbacks	LangSmith, Console, Custom	Logging, tracing, monitoring

1.4 Installation & Setup

Install LangChain and required dependencies using pip:

```
# Install core LangChain
pip install langchain langchain-core langchain-community

# Install OpenAI integration
pip install langchain-openai

# Install vector store support
pip install chromadb faiss-cpu

# Set API keys (use .env in production)
export OPENAI_API_KEY='sk-...'
```

1.5 First LangChain Program

A minimal but complete LangChain program — an LLM call with a prompt template:

```
from langchain_openai import ChatOpenAI
from langchain_core.prompts import ChatPromptTemplate
from langchain_core.output_parsers import StrOutputParser

# 1. Define the model
llm = ChatOpenAI(model='gpt-4o-mini', temperature=0.7)

# 2. Define the prompt
prompt = ChatPromptTemplate.from_messages([
    ('system', 'You are a helpful assistant.'),
    ('human', '{question}'),
])

# 3. Build the chain (LCEL pipe operator)
chain = prompt | llm | StrOutputParser()

# 4. Invoke
response = chain.invoke({'question': 'What is LangChain?'})
print(response)
```

■ *Note: The | pipe operator is LangChain Expression Language (LCEL). Each component is a Runnable that passes output to the next stage.*

Module 2 — LangChain Components

LangChain is built around a set of interoperable components. Each component implements the Runnable interface, meaning it can be composed into chains using LCEL. This section covers the most important components every developer must know.

2.1 Models — LLMs & Chat Models

Models are the foundation of any LangChain application. LangChain distinguishes between two types: LLMs (text-in / text-out) and Chat Models (messages-in / message-out). Modern applications almost exclusively use Chat Models.

```
from langchain_openai import OpenAI, ChatOpenAI
from langchain_anthropic import ChatAnthropic
from langchain_community.llms import Ollama

# Legacy LLM (text completion)
llm = OpenAI(model='gpt-3.5-turbo-instruct', temperature=0)

# Chat Model (recommended for most use cases)
chat = ChatOpenAI(model='gpt-4o', temperature=0.5, max_tokens=1024)

# Anthropic Claude
claude = ChatAnthropic(model='claude-3-5-sonnet-20241022')

# Local model via Ollama (free, offline)
local = Ollama(model='llama3.2')

# Invoke
result = chat.invoke('Tell me about LangChain in two sentences.')
print(result.content)
```

2.2 Messages

Chat models communicate via typed messages. LangChain defines four message types:

Message Type	Role	Usage
SystemMessage	system	Sets LLM persona/instructions for the entire conversation
HumanMessage	user	Input from the user / end-user

Message Type	Role	Usage
AIMessage	assistant	Response generated by the LLM
ToolMessage	tool	Result returned from a tool/function call

2.3 Output Parsers

Output parsers transform raw LLM text into structured Python objects. This is critical when your downstream code needs a list, dict, or Pydantic model rather than free-form text.

```
from langchain_core.output_parsers import StrOutputParser, JsonOutputParser
from langchain_core.pydantic_v1 import BaseModel, Field

# String parser — strips message wrapper
str_parser = StrOutputParser()

# JSON parser — returns dict
json_parser = JsonOutputParser()

# Structured parser with Pydantic
class MovieReview(BaseModel):
    title: str = Field(description='Movie title')
    rating: int = Field(description='Rating 1-10')
    summary: str = Field(description='One-sentence summary')

structured_parser = JsonOutputParser(pydantic_object=MovieReview)

chain = prompt | llm | structured_parser
review = chain.invoke({'movie': 'Inception'})
print(review['title'], review['rating'])
```

2.4 Document Loaders & Text Splitters

Document loaders import external data — PDFs, web pages, databases — into LangChain's Document format. Text splitters then chunk long documents into pieces suitable for embedding and retrieval.

```
from langchain_community.document_loaders import PyPDFLoader, WebBaseLoader
from langchain_text_splitters import RecursiveCharacterTextSplitter

# Load a PDF
loader = PyPDFLoader('report.pdf')
docs = loader.load()   # returns List[Document]

# Load a web page
web_loader = WebBaseLoader('https://python.langchain.com/docs/')
web_docs = web_loader.load()

# Split into chunks
splitter = RecursiveCharacterTextSplitter(
    chunk_size=1000,     # characters per chunk
    chunk_overlap=200,   # overlap for context continuity
)
chunks = splitter.split_documents(docs)
print(f'Split into {len(chunks)} chunks')
```

2.5 Embeddings & Vector Stores

Embeddings convert text into numerical vectors that capture semantic meaning. Vector stores index these vectors for fast similarity search — the backbone of RAG (Retrieval-Augmented Generation) pipelines.

```
from langchain_openai import OpenAIEmbeddings
from langchain_community.vectorstores import Chroma, FAISS

# Create embedding model
embeddings = OpenAIEmbeddings(model='text-embedding-3-small')

# Build vector store from documents
vectorstore = Chroma.from_documents(
    documents=chunks,
    embedding=embeddings,
    persist_directory='./chroma_db'
)

# Similarity search
results = vectorstore.similarity_search('What is LangChain?', k=3)
for doc in results:
    print(doc.page_content[:200])
```

Module 3 — Chains

Chains are the core concept in LangChain. A chain is a sequence of calls to components — models, prompts, retrievers, tools — where the output of one step becomes the input of the next. Chains can be simple (a single LLM call) or complex (multi-step reasoning with tool use and retrieval).

3.1 What are Chains?

Prior to LangChain Expression Language (LCEL), chains were created using class-based objects. Today, LCEL is the recommended approach. Every Runnable exposes .invoke(), .stream(), .batch(), and .astream() — allowing sync, async, and streaming out of the box.

> **■ Chain Paradigm**
> Chain = Prompt | LLM | OutputParser. The | operator pipes output from left to right, creating a declarative, composable pipeline that is easy to read and easy to test.

3.2 LLMChain (Classic)

LLMChain is the simplest chain — it combines a prompt template with an LLM. While the class-based API still works, LCEL is preferred in new code.

```
from langchain.chains import LLMChain
from langchain_openai import ChatOpenAI
from langchain.prompts import PromptTemplate

# Classic LLMChain
llm = ChatOpenAI(model='gpt-4o-mini')
prompt = PromptTemplate(
    input_variables=['topic'],
    template='Explain {topic} in simple terms.'
)
chain = LLMChain(llm=llm, prompt=prompt)
result = chain.run(topic='neural networks')
print(result)

# LCEL equivalent (modern, preferred)
from langchain_core.output_parsers import StrOutputParser
chain_lcel = prompt | llm | StrOutputParser()
result2 = chain_lcel.invoke({'topic': 'neural networks'})
```

3.3 Sequential Chains

Sequential chains connect multiple LLM calls where the output of the first call feeds into the second. This enables multi-step reasoning — for example, first generating a draft, then critiquing it, then refining it.

```
from langchain_core.prompts import ChatPromptTemplate
from langchain_openai import ChatOpenAI
from langchain_core.output_parsers import StrOutputParser

llm = ChatOpenAI(model='gpt-4o-mini')
parser = StrOutputParser()

# Step 1: Write a joke
joke_prompt = ChatPromptTemplate.from_template(
    'Write a short joke about {topic}.'
)
joke_chain = joke_prompt | llm | parser

# Step 2: Explain why it is funny
explain_prompt = ChatPromptTemplate.from_template(
    'Explain why this joke is funny: {joke}'
)
explain_chain = explain_prompt | llm | parser

# Compose: output of joke_chain becomes input of explain_chain
full_chain = joke_chain | (lambda joke: {'joke': joke}) | explain_chain
result = full_chain.invoke({'topic': 'Python programming'})
print(result)
```

3.4 Router Chains

Router chains route input to different sub-chains based on the input content. They are useful when you have specialized chains for different topics or task types and want to automatically select the best one.

```
from langchain_core.runnables import RunnableBranch
from langchain_core.output_parsers import StrOutputParser

# Define specialized chains
math_chain   = math_prompt   | llm | StrOutputParser()
history_chain = history_prompt | llm | StrOutputParser()
default_chain = default_prompt | llm | StrOutputParser()

# Build router using RunnableBranch
router = RunnableBranch(
    (lambda x: 'math'    in x['topic'].lower(), math_chain),
    (lambda x: 'history' in x['topic'].lower(), history_chain),
    default_chain   # fallback
)

result = router.invoke({'topic': 'math equations', 'question': '...'})
```

3.5 LCEL — LangChain Expression Language

LCEL is the modern, recommended way to build chains. It uses Python's | operator to compose Runnables. LCEL chains natively support streaming, async, batch processing, input/output schema validation, and LangSmith tracing.

LCEL Feature	How to Use	Benefit
Streaming	chain.stream(input)	Token-by-token output for UIs
Async	await chain.ainvoke(input)	Non-blocking for async apps
Batch	chain.batch([in1, in2, in3])	Parallel processing of multiple inputs
Fallbacks	chain.with_fallbacks([chain2])	Automatic retry with backup chain
Retry	chain.with_retry(max_retries=3)	Resilient chains in production
Passthrough	RunnablePassthrough()	Forward input unchanged

```
from langchain_core.runnables import RunnablePassthrough, RunnableParallel

# Parallel execution — run two chains simultaneously
parallel_chain = RunnableParallel({
    'english': prompt_en | llm | StrOutputParser(),
    'french':  prompt_fr | llm | StrOutputParser(),
})
result = parallel_chain.invoke({'text': 'Hello world'})

# Streaming example
for chunk in chain.stream({'question': 'Tell me a story'}):
    print(chunk, end='', flush=True)
```

Module 4 — Prompt Templates

Prompts are the primary interface through which developers communicate intent to an LLM. A PromptTemplate is a parameterised string or message sequence with named variables that get filled in at runtime. Good prompt engineering is the difference between a mediocre and an excellent LLM application.

4.1 PromptTemplate (String Prompts)

The simplest template — a single string with {variable} placeholders. Use this for LLMs (text completion models) or when you need a single formatted string.

```
from langchain_core.prompts import PromptTemplate

# Define template with variables
template = PromptTemplate(
    input_variables=['product', 'audience'],
    template='Write a compelling tagline for {product} targeting {audience}.'
)

# Format the prompt (returns a string)
formatted = template.format(product='LangChain', audience='developers')
print(formatted)
# Output: Write a compelling tagline for LangChain targeting developers.

# Shorthand using from_template
template2 = PromptTemplate.from_template(
    'Summarise the following in {n} bullet points: {text}'
)
```

4.2 ChatPromptTemplate

ChatPromptTemplate produces a list of typed messages rather than a single string. This is the correct template type for ChatOpenAI, ChatAnthropic, and virtually all modern LLM integrations. It supports system, human, and AI message roles.

```python
from langchain_core.prompts import ChatPromptTemplate, MessagesPlaceholder

# Basic system + human template
prompt = ChatPromptTemplate.from_messages([
    ('system', 'You are a {role}. Respond formally.'),
    ('human',  '{question}'),
])

messages = prompt.format_messages(role='data scientist', question='What is RAG?')

# With conversation history placeholder
chat_prompt = ChatPromptTemplate.from_messages([
    ('system', 'You are a helpful assistant.'),
    MessagesPlaceholder(variable_name='history'),  # inject memory here
    ('human', '{input}'),
])
```

■ MessagesPlaceholder

MessagesPlaceholder is the key to injecting conversation memory into a ChatPromptTemplate. Pass a list of HumanMessage and AIMessage objects to the history variable at runtime.

4.3 FewShotPromptTemplate

Few-shot prompting includes worked examples in the prompt to guide the LLM's output format and reasoning style. FewShotPromptTemplate automates the formatting of these examples, including dynamic example selection based on input similarity.

```
from langchain_core.prompts import FewShotPromptTemplate, PromptTemplate

# Define examples
examples = [
    {'word': 'happy',   'antonym': 'sad'},
    {'word': 'tall',    'antonym': 'short'},
    {'word': 'energetic', 'antonym': 'lethargic'},
]

# Template for each example
example_prompt = PromptTemplate(
    input_variables=['word', 'antonym'],
    template='Word: {word}\nAntonym: {antonym}'
)

# Few-shot template
few_shot = FewShotPromptTemplate(
    examples=examples,
    example_prompt=example_prompt,
    prefix='Give the antonym of each word.',
    suffix='Word: {input}\nAntonym:',
    input_variables=['input']
)
print(few_shot.format(input='brave'))
```

4.4 Partial Variables & Template Composition

Partial variables pre-fill some template variables at creation time, leaving others for runtime. Template composition allows building complex prompts from reusable pieces.

```
from datetime import datetime

# Partial with a static value
prompt = PromptTemplate(
    input_variables=['topic', 'date'],
    template='As of {date}, explain {topic}.'
)
prompt_partial = prompt.partial(date=datetime.now().strftime('%Y-%m-%d'))
# Now only 'topic' is required at runtime
result = prompt_partial.format(topic='AI trends')

# Partial with a callable (evaluated at runtime)
prompt_dynamic = prompt.partial(date=lambda: datetime.now().strftime('%Y-%m-%d'))
```

Module 5 — Memory Systems Overview

LLMs are stateless by design — each API call is independent. Without memory, every user turn starts from scratch. Memory systems solve this by storing and injecting conversation history (or summaries) into subsequent prompts, giving the LLM the illusion of continuity.

5.1 Why Memory Matters

- **Context continuity:** The LLM can refer to earlier parts of the conversation.
- **Personalisation:** Remember user preferences, name, and prior requests.
- **Multi-turn tasks:** Complex tasks that span multiple exchanges (e.g., writing an essay step by step).
- **Coherent assistants:** Prevent the LLM from contradicting itself across turns.

5.2 The Memory Interface

All LangChain memory classes share a common interface with two key methods: load_memory_variables() which returns context to inject into prompts, and save_context() which stores the latest input/output pair.

Memory Type	Storage	Best For	Token Cost
ConversationBufferMemory	All messages	Short conversations	High (grows linearly)
ConversationBufferWindowMemory	Last K messages	Medium conversations	Bounded
ConversationSummaryMemory	LLM-generated summary	Long conversations	Low (fixed)
ConversationSummaryBufferMemory	Summary + recent buffer	Balanced approach	Medium
VectorStoreRetrieverMemory	Vector DB	Long-term / semantic recall	Very low
ConversationEntityMemory	Entity facts	Tracking people/things	Low

5.3 Memory in Chains

Memory is integrated into chains by injecting stored context into the prompt via a MessagesPlaceholder. The ConversationChain class handles this automatically; with LCEL you wire it up manually using RunnableWithMessageHistory.

```
from langchain.chains import ConversationChain
from langchain.memory import ConversationBufferMemory
from langchain_openai import ChatOpenAI

llm = ChatOpenAI(model='gpt-4o-mini')
memory = ConversationBufferMemory()

conversation = ConversationChain(
    llm=llm,
    memory=memory,
    verbose=True   # show the prompt being built
)

conversation.predict(input='Hi, my name is Neeraj.')
conversation.predict(input='What is my name?')   # LLM recalls: Neeraj
```

Module 6 — Conversation Memory

Conversation memory stores the dialogue between the user and the assistant. LangChain provides multiple variants depending on your token budget and the length of conversations you expect.

6.1 ConversationBufferMemory

The simplest memory type — stores every message in the conversation verbatim. The complete history is injected into each new prompt. Ideal for short conversations but becomes expensive (and eventually hits context limits) for long ones.

```
from langchain.memory import ConversationBufferMemory
from langchain_core.messages import HumanMessage, AIMessage

memory = ConversationBufferMemory(
    return_messages=True,     # return Message objects (for Chat models)
    memory_key='history',     # key used in ChatPromptTemplate
)

# Manually save context
memory.save_context(
    {'input': 'Who invented Python?'},
    {'output': 'Guido van Rossum created Python in 1991.'}
)

# Load memory variables
context = memory.load_memory_variables({})
print(context['history'])   # list of HumanMessage + AIMessage

# Clear memory
memory.clear()
```

6.2 ConversationSummaryMemory

Instead of storing every message, ConversationSummaryMemory uses an LLM to progressively summarise the conversation. As the chat grows longer, older turns are compressed into a running summary, keeping token usage approximately constant.

```
from langchain.memory import ConversationSummaryMemory
from langchain_openai import ChatOpenAI

llm = ChatOpenAI(model='gpt-4o-mini')

memory = ConversationSummaryMemory(
    llm=llm,          # used internally to generate summaries
    return_messages=True
)

# After several turns, memory.buffer contains a summary rather than raw messages
memory.save_context({'input': 'Tell me about RAG.'}, {'output': '...'})
memory.save_context({'input': 'How is it different from fine-tuning?'}, {'output': '...'})

print(memory.buffer)   # concise LLM-generated summary
```

■■ Summary vs Buffer

Buffer memory is exact but expensive. Summary memory is approximate but cheap. For production chatbots with long sessions, use ConversationSummaryBufferMemory — it keeps recent messages verbatim and summarises older ones.

6.3 ConversationBufferWindowMemory

This variant keeps only the last K conversation turns in the buffer, discarding older turns. It provides a sliding window of context — useful when only recent context matters and you want predictable token costs.

```
from langchain.memory import ConversationBufferWindowMemory

# Keep only last 5 turns (human + AI = 1 turn)
memory = ConversationBufferWindowMemory(
    k=5,
    return_messages=True
)

# After 6 turns, the oldest turn is dropped automatically
for i in range(7):
    memory.save_context(
        {'input': f'Question {i+1}'},
        {'output': f'Answer {i+1}'}
    )

messages = memory.load_memory_variables({})['history']
print(f'Stored turns: {len(messages) // 2}')   # 5, not 7
```

6.4 Using Memory with LCEL (Modern Approach)

With LCEL chains, use RunnableWithMessageHistory to attach any BaseChatMessageHistory store (in-memory, Redis, DynamoDB, etc.) to your chain. This is the production-grade approach.

```
from langchain_core.runnables.history import RunnableWithMessageHistory
from langchain_community.chat_message_histories import ChatMessageHistory
from langchain_core.prompts import ChatPromptTemplate, MessagesPlaceholder
from langchain_openai import ChatOpenAI
from langchain_core.output_parsers import StrOutputParser

llm = ChatOpenAI(model='gpt-4o-mini')

prompt = ChatPromptTemplate.from_messages([
    ('system', 'You are a helpful assistant.'),
    MessagesPlaceholder(variable_name='history'),
    ('human', '{input}'),
])

chain = prompt | llm | StrOutputParser()

# Session store — keyed by session_id
store = {}
def get_session_history(session_id: str):
    if session_id not in store:
        store[session_id] = ChatMessageHistory()
    return store[session_id]

# Wrap chain with history management
chain_with_memory = RunnableWithMessageHistory(
    chain,
    get_session_history,
    input_messages_key='input',
    history_messages_key='history',
)

# Multi-turn conversation
config = {'configurable': {'session_id': 'user_123'}}
chain_with_memory.invoke({'input': 'My name is Neeraj.'}, config=config)
chain_with_memory.invoke({'input': 'What is my name?'},   config=config)
```

Module 7 — Buffer Memory Deep Dive

Buffer memory is the foundational memory mechanism in LangChain. All messages — both human turns and AI responses — are stored in a buffer (list) and replayed verbatim in every subsequent prompt. Understanding how it works, its limitations, and how to manage it is essential before moving to more advanced memory types.

7.1 How Buffer Memory Works Internally

The ChatMessageHistory object maintains a list of BaseMessage instances. Each call to save_context() appends a HumanMessage and an AIMessage. When load_memory_variables() is called, the complete list is returned under the configured key.

```
from langchain_community.chat_message_histories import ChatMessageHistory
from langchain_core.messages import HumanMessage, AIMessage

# Inspect the underlying store
history = ChatMessageHistory()
history.add_user_message('What is an LLM?')
history.add_ai_message('An LLM is a Large Language Model trained on text data.')
history.add_user_message('Give an example.')
history.add_ai_message('GPT-4 and Claude 3.5 are popular LLMs.')

for msg in history.messages:
    role = 'Human' if isinstance(msg, HumanMessage) else 'AI'
    print(f'[{role}]: {msg.content}')

# Total token estimate
total_chars = sum(len(m.content) for m in history.messages)
print(f'Approx tokens: {total_chars // 4}')
```

7.2 Token Limits & Context Windows

Every LLM has a maximum context window — the total number of tokens it can process in a single call, including the system prompt, history, user input, and output. As the buffer grows, you approach this limit.

Model	Context Window	Safe Buffer Size
gpt-4o-mini	128K tokens	~50 turns (depends on message length)

Model	Context Window	Safe Buffer Size
gpt-4o	128K tokens	~50 turns
claude-3-5-sonnet	200K tokens	~80 turns
llama3.2 (local)	128K tokens	~50 turns
gemini-1.5-flash	1M tokens	~400 turns

7.3 Pruning Strategies

When a buffer exceeds the safe limit, you have several options to reduce token count while preserving conversational coherence:

- **Window truncation:** Drop the oldest N turns (ConversationBufferWindowMemory).
- **Summarisation:** Replace old turns with an LLM-generated summary (ConversationSummaryMemory).
- **Hybrid buffer:** Summarise old turns, keep recent ones verbatim (ConversationSummaryBufferMemory).
- **Selective trimming:** Use trim_messages() to remove turns exceeding a token budget.
- **Compression:** Run old messages through a compression chain before storing.

```
from langchain_core.messages import trim_messages
from langchain_openai import ChatOpenAI

llm = ChatOpenAI(model='gpt-4o-mini')

# Trim messages to stay within 1000 tokens
trimmer = trim_messages(
    max_tokens=1000,
    strategy='last',          # keep the most recent messages
    token_counter=llm,        # use model's tokeniser for accuracy
    include_system=True,      # always keep the system message
    allow_partial=False,      # never split a message mid-way
    start_on='human',         # ensure buffer starts with a human turn
)

trimmed_messages = trimmer.invoke(history.messages)
print(f'Kept {len(trimmed_messages)} of {len(history.messages)} messages')
```

7.4 ConversationSummaryBufferMemory

The best-of-both-worlds memory type. It maintains a buffer of the most recent messages and a running summary of older ones. When the buffer exceeds max_token_limit, the oldest messages are summarised and moved into the summary string.

```
from langchain.memory import ConversationSummaryBufferMemory
from langchain_openai import ChatOpenAI

llm = ChatOpenAI(model='gpt-4o-mini')

memory = ConversationSummaryBufferMemory(
    llm=llm,
    max_token_limit=500,    # keep buffer under 500 tokens
    return_messages=True
)

# Simulate a long conversation
turns = [
    ('What is LangChain?', 'A framework for LLM apps.'),
    ('What are chains?',   'Sequences of LLM calls.'),
    ('What is memory?',    'Stores conversation history.'),
    ('What is LCEL?',      'LangChain Expression Language.'),
    ('What are agents?',   'LLMs that call tools dynamically.'),
]
for inp, out in turns:
    memory.save_context({'input': inp}, {'output': out})

# Print what is stored
print('Summary:', memory.moving_summary_buffer)
print('Recent messages:', len(memory.chat_memory.messages))
```

Module 8 — Introduction to Long-Term Memory

Short-term memory persists only within a single session. Long-term memory persists across sessions and enables an LLM to recall information from days, weeks, or months ago. LangChain provides several approaches to long-term memory, all based on persistent storage backends.

8.1 Vector Store Memory

VectorStoreRetrieverMemory stores conversation turns as embeddings in a vector database. Rather than injecting the entire history, it retrieves only the most semantically relevant past exchanges for the current question. This scales to millions of stored memories.

```
from langchain.memory import VectorStoreRetrieverMemory
from langchain_community.vectorstores import Chroma
from langchain_openai import OpenAIEmbeddings

# Set up vector store
embeddings = OpenAIEmbeddings()
vectorstore = Chroma(
    embedding_function=embeddings,
    persist_directory='./long_term_memory'   # persists to disk
)

retriever = vectorstore.as_retriever(search_kwargs={'k': 3})

# Create vector memory
memory = VectorStoreRetrieverMemory(retriever=retriever)

# Save facts from past sessions
memory.save_context(
    {'input': 'My favourite language is Python.'},
    {'output': 'Noted! I will remember that.'}
)
memory.save_context(
    {'input': 'I am working on a RAG chatbot project.'},
    {'output': 'Great! RAG is a powerful pattern.'}
)

# Next session — retrieve relevant memories
relevant = memory.load_memory_variables({'prompt': 'What am I building?'})
print(relevant['history'])   # recalls the RAG chatbot conversation
```

8.2 Entity Memory

ConversationEntityMemory tracks facts about specific named entities — people, places, products, organisations — as the conversation progresses. An LLM is used to extract and update entity summaries from each exchange.

```python
from langchain.memory import ConversationEntityMemory
from langchain_openai import ChatOpenAI

llm = ChatOpenAI(model='gpt-4o-mini')
memory = ConversationEntityMemory(llm=llm)

memory.save_context(
    {'input': 'Neeraj is a trainer who teaches LangChain bootcamps.'},
    {'output': 'That is great! LangChain is a powerful framework.'}
)
memory.save_context(
    {'input': 'Neeraj uses Jaipur as his training base.'},
    {'output': 'Jaipur is a great city for tech education!'}
)

# Entity store now contains facts about 'Neeraj' and 'Jaipur'
print(memory.entity_store.store)
# {'Neeraj': 'A trainer who teaches LangChain bootcamps in Jaipur.',
#  'Jaipur': 'Training base for Neeraj, a LangChain trainer.'}
```

8.3 Persistent Backends for Long-Term Memory

For production systems, memory must survive server restarts. LangChain supports many backends for persistent message storage:

Backend	Package	Best For
In-Memory (default)	langchain-core	Development / testing only
SQLite	langchain-community	Small-scale single-user apps
PostgreSQL	langchain-postgres	Production multi-user apps
Redis	langchain-redis	High-throughput / low-latency apps
DynamoDB	langchain-aws	Serverless AWS deployments
MongoDB	langchain-mongodb	Document-heavy applications

```
# Example: SQLite persistence across sessions
from langchain_community.chat_message_histories import SQLChatMessageHistory

def get_session_history(session_id: str):
    return SQLChatMessageHistory(
        session_id=session_id,
        connection_string='sqlite:///memory.db'
    )

# Wrap chain with persistent history
chain_with_history = RunnableWithMessageHistory(
    chain,
    get_session_history,
    input_messages_key='input',
    history_messages_key='history',
)

# Session survives restarts — history loaded from SQLite
chain_with_history.invoke(
    {'input': 'Continue from where we left off.'},
    config={'configurable': {'session_id': 'user_123'}}
)
```

8.4 Combining Memory Types

Sophisticated assistants often combine multiple memory types. A common production pattern is: short-term buffer memory for recent turns + vector store for semantic long-term recall + entity memory for structured facts about users.

Memory Layer	What it Stores	Retrieval Method
ConversationBufferWindowMemory (k=10)	Last 10 turns verbatim	Always injected
VectorStoreRetrieverMemory	All past sessions as embeddings	Semantic similarity search
ConversationEntityMemory	Structured facts per entity	Entity name lookup

■■ Production Architecture

A production chatbot typically uses: Redis-backed ChatMessageHistory for the session buffer, Chroma or Pinecone for long-term vector memory, and a PostgreSQL entity store for user profile facts. LangSmith traces every chain call for debugging.

8.5 Memory Quick-Reference Summary

Memory Class	Persists Across Sessions?	Token Cost	Recommended When
ConversationBufferMemory	No	High	Short demos, prototypes
ConversationBufferWindowMemory	No	Bounded	Medium sessions, predictable cost
ConversationSummaryMemory	No	Low	Long sessions, less precision needed
ConversationSummaryBufferMemory	No	Medium	Best balance for production
VectorStoreRetrieverMemory	Yes (with persistent DB)	Very Low	Long-term recall, large history
ConversationEntityMemory	Yes (if backed by DB)	Low	Tracking user facts & entities
SQLChatMessageHistory	Yes	Raw messages	Full history with persistence

Ready to Build?

You now have a complete foundation in LangChain — from raw LLM calls and prompt templates
through chains and all major memory types. The next step is to build a full RAG chatbot
combining retrieval, LCEL chains, and persistent long-term memory.

■ LangChain Foundations | Bootcamp Training Series | 2025